Introduction to Linked list

Lesson Plan



PW SKILLS

Today's Checklist:

- Limitations of Arrays
- Introduction to Linked List
- Implementing Linked List
- Displaying
- Insert in Linked List
- Limitation
- Delete in Linked List

Limitations of Arrays

- **Fixed Size:** Most arrays have a fixed size, meaning you need to know the number of elements in advance. This can be a limitation when the size of the data is dynamic and unknown beforehand.

- **Contiguous Memory Allocation:** Elements in an array are stored in contiguous memory locations. This can lead to fragmentation and might make it difficult to find a large enough block of memory for the array.

- **Inefficient Insertions and Deletions:** Inserting or deleting elements in the middle of an array requires shifting all subsequent elements, which can be inefficient. The time complexity for these operations is O(n), where n is the number of elements.

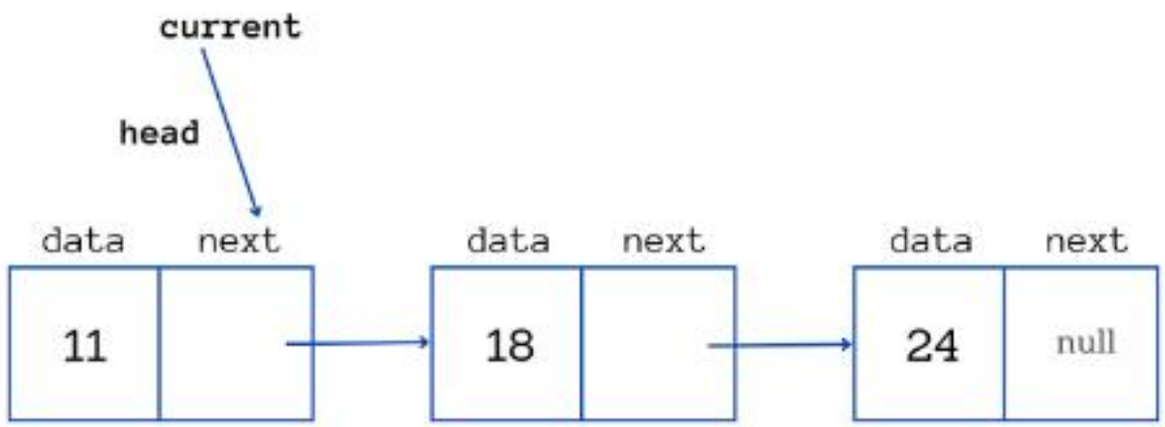
- **Wastage of Memory:** If you allocate more space than needed for an array, you may end up wasting memory. This is particularly problematic when the array size is predetermined to accommodate the worst-case scenario.

- **Homogeneous Data Types:** Arrays typically store elements of the same data type. This can be limiting when you need to store elements of different types.

- **Memory Fragmentation:** The contiguous memory allocation can lead to memory fragmentation, making it challenging to allocate a large contiguous block of memory for a new array.

Introduction to Linked List

It is basically chains of nodes, each node contains information such as data and a pointer to the next node in the chain. In the linked list there is a head pointer, which points to the first element of the linked list, and if the list is empty then it simply points to null or nothing.



Why linked list data structure needed?

- Dynamic Data structure: The size of memory can be allocated or de-allocated at run time based on the operation insertion or deletion.

- Ease of Insertion/Deletion: The insertion and deletion of elements are simpler than arrays since no elements need to be shifted after insertion and deletion, Just the address needed to be updated.

- Efficient Memory Utilization: As we know Linked List is a dynamic data structure the size increases or decreases as per the requirement so this avoids the wastage of memory.

- Implementation: Various advanced data structures can be implemented using a linked list like a stack, queue, graph, hash maps, etc.

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

Implementing Linked List

A linked list can be implemented in various ways, but the basic structure involves nodes, where each node contains data and a reference (or link) to the next node in the sequence.

A step-by-step explanation of how to implement a simple singly linked list:

- **Node Class:**

Create a class for the linked list node with data and a pointer to the next node.

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None
```

- **LinkedList Class:**

Create a class to represent the linked list.
Include a pointer to the head of the list.

```
class LinkedList:
    def __init__(self):
        self.head = None
```

- **Add Nodes:**
- Implement a method to add nodes to the linked list.
- If the list is empty, create a new node and set it as the head.
- Otherwise, traverse to the end of the list and add a new node.

```
def add_node(self, value):
    new_node = Node(value)
    if self.head is None:
        self.head = new_node
    else:
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node
```

- **Displaying**

Once, we have created the linked list, we would like to see the elements inside it. Displaying a linked list involves iterating through its nodes and printing their data. This can also be done recursively as shown in the code below.

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None

def display_linked_list_iterative(head):
    while head is not None:
        print(head.data, end=" ")
        head = head.next
    print()

def display_linked_list_recursive(head):
    if head is None:
        return
    print(head.data, end=" ")
    display_linked_list_recursive(head.next)

if __name__ == "__main__":
    head = Node(1)
    head.next = Node(2)
    head.next.next = Node(3)
    head.next.next.next = Node(4)

    print("Iterative Display: ", end="")
    display_linked_list_iterative(head)

    print("Recursive Display: ", end="")
    display_linked_list_recursive(head)
    print()
```

Explanation:

Iterative Display:

- Initialize a pointer to the head of the linked list.
- Use a while loop to traverse the list.
- Print the data of each node and move the pointer to the next node.
- Repeat until the end of the list is reached.

Recursive Display:

- Base case: If the current node is null, return.
- Print the data of the current node.
- Make a recursive call with the next node.
- The recursion unwinds, printing nodes in sequential order.
- Base case ensures termination when the end of the list is reached.

Iterative Display:

Time Complexity: O(n) - where 'n' is the number of nodes in the linked list. The algorithm iterates through each node once.
Space Complexity: O(1) - uses a constant amount of extra space, regardless of the size of the linked list.

Recursive Display:

Time Complexity: O(n) - where 'n' is the number of nodes in the linked list. Similar to the iterative approach, each node is visited once, but the recursive call stack contributes to the time complexity.
Space Complexity: O(n) - due to the recursive call stack. The maximum depth of the recursion is 'n', corresponding to the length of the linked list.

Length of Linked List

```
def find_length_iterative(head):
    length = 0
    while head is not None:
        length += 1
        head = head.next
    return length

# Test the iterative length function
if __name__ == "__main__":
    head = Node(1)
    head.next = Node(2)
    head.next.next = Node(3)
    head.next.next.next = Node(4)
    print("Length (Iterative):", find_length_iterative(head))  #
Output: 4


def find_length_recursive(head):
    if head is None:
        return 0
    return 1 + find_length_recursive(head.next)

# Test the recursive length function
if __name__ == "__main__":
    head = Node(1)
    head.next = Node(2)
    head.next.next = Node(3)
    head.next.next.next = Node(4)
    print("Length (Recursive):", find_length_recursive(head))  #
Output: 4
```

Iterative Method:
Time Complexity: O(n) - where 'n' is the number of nodes in the linked list. In the worst case, it needs to traverse all nodes once.

Space Complexity: O(1) - uses a constant amount of extra space.

Recursive Method:
Time Complexity: O(n) - where 'n' is the number of nodes in the linked list. Similar to the iterative approach, it needs to visit each node once.

Space Complexity: O(n) - due to the recursive call stack. The maximum depth of the recursion is 'n', corresponding to the length of the linked list.

Insert in Linked List

- The insertion operation can be performed in three ways. They are as follows...
- Inserting At the Beginning of the list
- Inserting At End of the list
- Inserting At Specific location in the list

Code:

```
def insert_at_beginning(head, value):
    new_node = Node(value)
    new_node.next = head
    return new_node

# Test insert at beginning
if __name__ == "__main__":
    head = None
    head = insert_at_beginning(head, 1)
    head = insert_at_beginning(head, 2)
    display(head)  # Output: 2 1

def insert_at_end(head, value):
    new_node = Node(value)
    if head is None:
        return new_node
    current = head
    while current.next is not None:
        current = current.next
    current.next = new_node
    return head

# Test insert at end
if __name__ == "__main__":
    head = None
    head = insert_at_end(head, 1)
    head = insert_at_end(head, 2)
    display(head)  # Output: 1 2

def insert_at_location(head, value, position):
    new_node = Node(value)
    if position == 1:
        new_node.next = head
        return new_node
    current = head
    for i in range(1, position - 1):
        if current is None:
            print("Invalid position.")
            return head
        current = current.next
    if current is None:
        print("Invalid position.")
        return head
    new_node.next = current.next
    current.next = new_node
    return head

# Test insert at specific location
if __name__ == "__main__":
    head = None
    head = insert_at_end(head, 1)
    head = insert_at_end(head, 3)
    head = insert_at_location(head, 2, 2)
    display(head)  # Output: 1 2 3
```

Insert at the Beginning:

- Create a New Node: Allocate memory for a new node and set its data.
- Link to Current Head: Set the next pointer of the new node to the current head.
- Update Head: Set the new node as the new head of the linked list.

Insert at the End:

- Create a New Node: Allocate memory for a new node and set its data.
- Traverse to the Last Node: Iterate through the linked list until the last node is reached.
- Link to Last Node: Set the next pointer of the last node to the new node.

Insert at Specific Location:

- Create a New Node: Allocate memory for a new node and set its data.
- Handle Special Case (Insert at the Beginning): If the position is 1, link the new node to the current head and update the head.
- Traverse to the Previous Node: Iterate through the list to the node preceding the desired position.
- Link the New Node: Set the next pointer of the new node to the next node of the previous node, and set the next pointer of the previous node to the new node.

Time and Space Complexity:

1. **Insert at the Beginning:**

- Time Complexity: O(1)
- Space Complexity: O(1)

2. **Insert at the End:**

- Time Complexity: O(n) - in the worst case
- Space Complexity: O(1)

3. **Insert at Specific Location:**

- Time Complexity: O(position) - in the worst case
- Space Complexity: O(1)

Delete in Linked List

The deletion operation can be performed in three ways. They are as follows:

- Deleting from the Beginning of the list
- Deleting from the End of the list
- Deleting a Specific Node

Code:

```
def delete_from_beginning(head):
    if head is None:
        print("List is empty. Cannot delete.")
        return None
    new_head = head.next
    head.next = None
    return new_head

# Test delete from beginning
if __name__ == "__main__":
    head = None
    head = insert_at_end(head, 1)
    head = insert_at_end(head, 2)
    head = delete_from_beginning(head)
    display(head)  # Output: 2

def delete_from_end(head):
    if head is None:
        print("List is empty. Cannot delete.")
        return None
    if head.next is None:
        return None
    current = head
    while current.next.next is not None:
        current = current.next
    current.next = None
    return head

# Test delete from end
if __name__ == "__main__":
    head = None
    head = insert_at_end(head, 1)
    head = insert_at_end(head, 2)
    head = delete_from_end(head)
    display(head)  # Output: 1

def delete_node(head, value):
    if head is None:
        print("List is empty. Cannot delete.")
        return None
    if head.data == value:
        new_head = head.next
        head.next = None
        return new_head
    current = head
    while current.next is not None and current.next.data ≠
value:
        current = current.next
    if current.next is None:
        print(f"Node with value {value} not found.")
        return head
    temp = current.next
    current.next = current.next.next
    temp.next = None
    return head

# Test delete specific node
if __name__ == "__main__":
    head = None
    head = insert_at_end(head, 1)
    head = insert_at_end(head, 2)
    head = insert_at_end(head, 3)
    head = delete_node(head, 2)
    display(head)  # Output: 1 3
```

Explanation:

Delete from the Beginning:
Check if the list is empty. If not, delete the current head and set the next node as the new head.

Delete from the End:
Check if the list is empty or has only one node. If not, traverse to the second-to-last node, delete the last node, and set the next pointer of the second-to-last node to null.

Delete a Specific Node:
Check if the list is empty. If not, traverse the list to find the node with the specified value. Delete the node by adjusting pointers.

Time and Space Complexity:

1. **Delete from the Beginning:**
   - Time Complexity: O(1)
   - Space Complexity: O(1)

2. **Delete from the End:**
   - Time Complexity: O(n) - in the worst case
   - Space Complexity: O(1)
   -

3. **Delete a Specific Node:**
   - Time Complexity: O(n) - in the worst case
   - Space Complexity: O(1)

Linked List-2

Lesson Plan



PW SKILLS

Today's Checklist:

1. Delete Node in a Linked List (Leetcode-237)
2. Middle of Linked List (Leetcode-876)
3. Remove Nth Node from End of List (Leetcode-19)
4. Intersection of two Linked Lists (Leetcode-160)
5. Linked List Cycle (Leetcode-141)
6. Linked List Cycle-II (Leetcode-142)

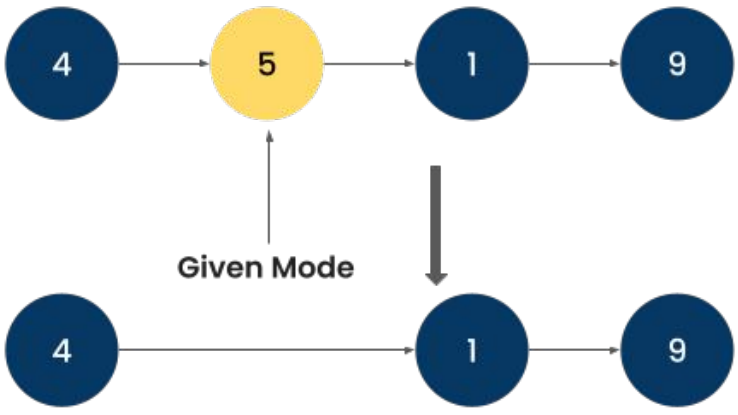
Delete Node in a Linked List (Leetcode-237)

There is a singly-linked list head and we want to delete a node node in it.
You are given the node to be deleted node. You will not be given access to the first node of head.
All the values of the linked list are unique, and it is guaranteed that the given node node is not the last node in the linked list.

Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before node should be in the same order.
- All the values after node should be in the same order.

Input: head = [4,5,1,9], node = 5
Output: [4,1,9]
Explanation: You are given the second node with value 5, the linked list should become 4 -> 1 -> 9 after calling your function.



Code:

```
class ListNode:
    def __init__(self, x):
        self.val = x
        self.next = None

class Solution:
    def deleteNode(self, node: ListNode) -> None:
        if node is not None and node.next is not None:
            node.val = node.next.val
            node.next = node.next.next
```

Time complexity: O(m + n), where m and n are the lengths of the two linked lists. The pointers traverse the linked lists once, and the loop continues until either the intersection is found or both pointers reach the end.

Space complexity: O(1). The algorithm uses only a constant amount of extra space for the two pointers (a and b), regardless of the size of the linked lists.

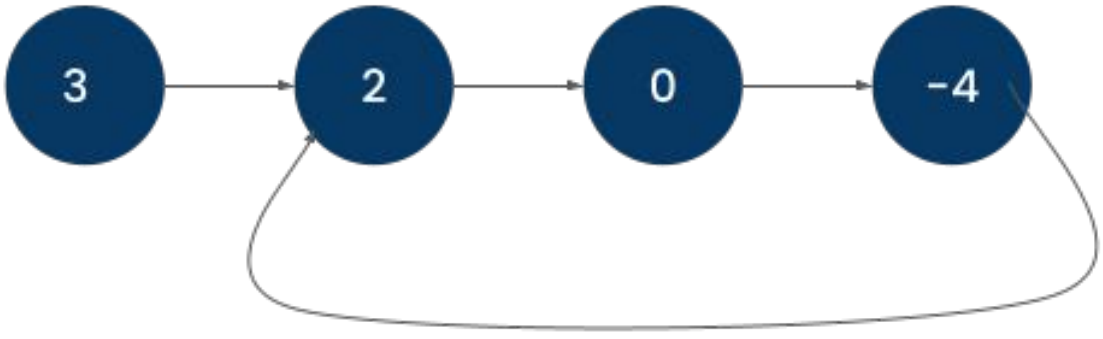
Linked List Cycle (Leetcode-141)

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

Return true if there is a cycle in the linked list. Otherwise, return false.

Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).



Code:

```
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head
        while fast is not None and fast.next is not None:
            slow = slow.next
            fast = fast.next.next
        return slow
```

Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm has at most two pointers traversing the linked list, and the loop will continue until the fast pointer reaches the end or the two pointers meet in a cycle.

Space complexity: O(1),The algorithm uses only a constant amount of extra space for the two pointers (slow_pointer and fast_pointer), regardless of the size of the linked list.

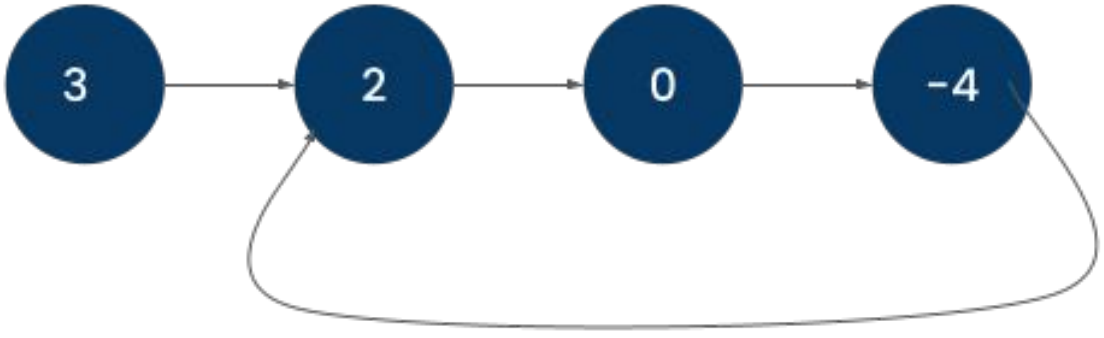
Linked List Cycle-II (Leetcode-142)

Given the head of a linked list, return the node where the cycle begins. If there is no cycle, return null.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to (0-indexed). It is -1 if there is no cycle. Note that pos is not passed as a parameter.

Do not modify the linked list.

Input: head = [3,2,0,-4], pos = 1
Output: tail connects to node index 1
Explanation: There is a cycle in the linked list, where tail connects to the second node.



Code:

```
lass Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) → ListNode:
        fast = head
        slow = head

        for _ in range(n):
            fast = fast.next

        if fast is None:
            return head.next

        while fast.next is not None:
            fast = fast.next
            slow = slow.next

        slow.next = slow.next.next

        return head
```

Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm uses two pointers, slow and fast, to traverse the linked list. In the worst case, it needs to iterate through the entire linked list once.

Space complexity: O(1),The algorithm uses only a constant amount of extra space for the two pointers (slow and fast), regardless of the size of the linked list. It doesn't use any additional data structures that scale with the input size.

Linked List-3

Lesson Plan



PW SKILLS

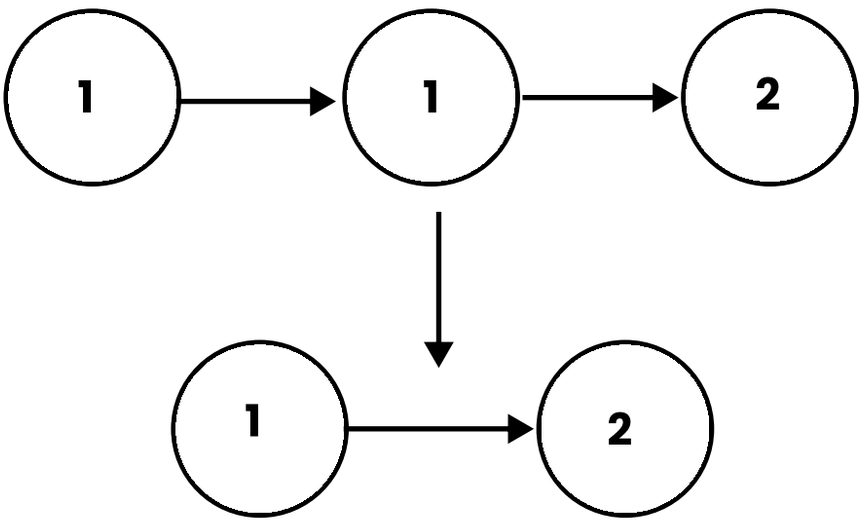
Today's Checklist:

1. Remove Duplicates from the Sorted List (Leetcode-83)
2. Rotate List (Leetcode-61)
3. Spiral Matrix IV (Leetcode-2326)
4. Merge 2 sorted lists (Leetcode-21)
5. Merge k sorted lists (Leetcode-23)
6. Sort List (Leetcode-148)
7. Partition List (Leetcode-86)
8. Reverse Linked List (Leetcode-206)
9. Palindrome Linked List (Leetcode-234)
10. Reverse Linked List II (Leetcode-92)
11. Reorder List (Leetcode-143)

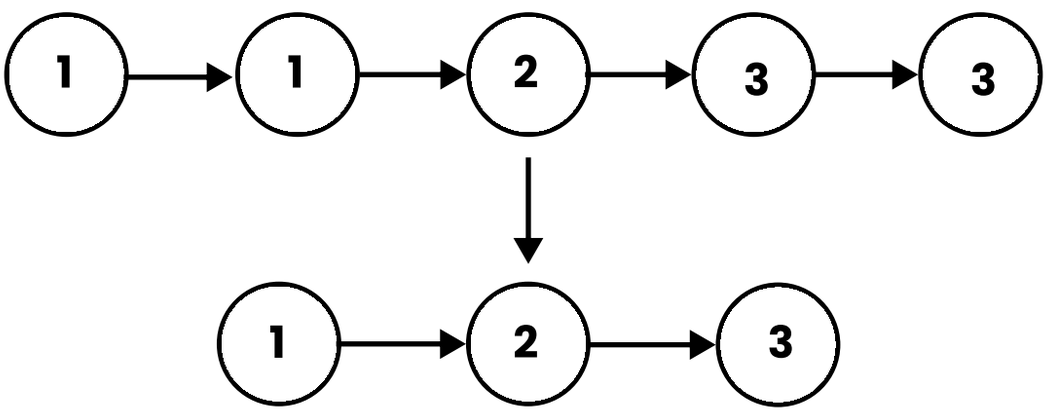
Remove Duplicates from the Sorted List (Leetcode-83)

Given the head of a sorted linked list, delete all duplicates such that each element appears only once. Return the linked list sorted as well.

Input: head = [1,1,2]
Output: [1,2]



Input: head = [1,1,2,3,3]
Output: [1,2,3]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteDuplicates(self, head: ListNode) -> ListNode:
        if not head or not head.next:
            return head

        slow = head
        fast = head.next

        while fast:
            if slow.val == fast.val:
                slow.next = fast.next
            else:
                slow = slow.next
            fast = fast.next
 return head
```

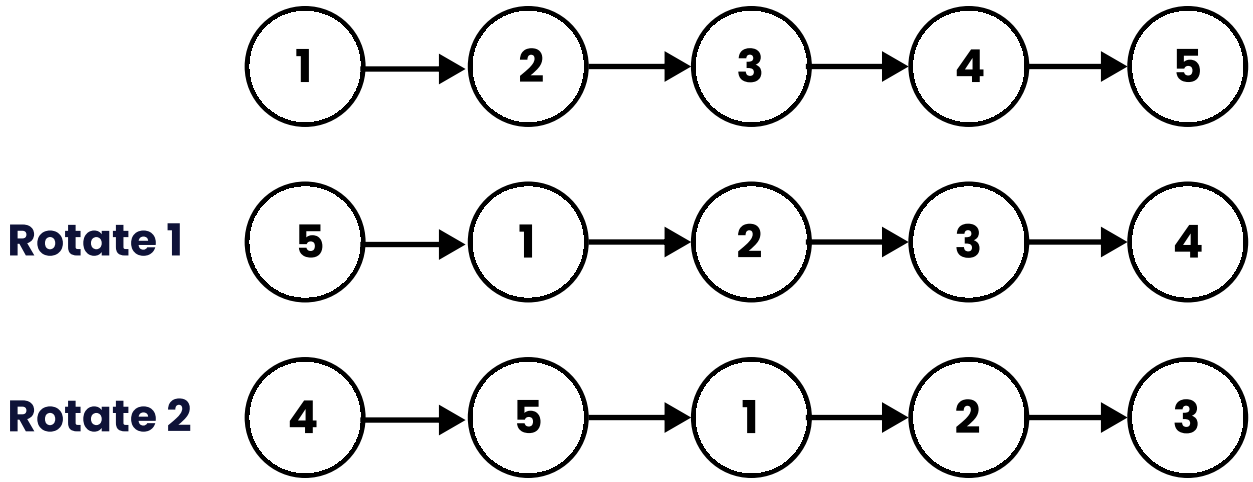
Time complexity: O(n) - where n is the number of nodes in the linked list.

Space complexity: O(1) - constant space is used, as no additional data structures are employed in the algorithm.

Rotate List (Leetcode-61)

Given the head of a linked list, rotate the list to the right by k places.

Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head:
            return head

        # Compute the length of the list
        length = 1
        tail = head
        while tail.next:
            tail = tail.next
            length += 1

        # Find the new head
        k = k % length
        if k == 0:
            return head

        # Move to the (length - k)th node
        new_tail = head
        for _ in range(length - k - 1):
            new_tail = new_tail.next

        new_head = new_tail.next
        new_tail.next = None
        tail.next = head

        return new_head
```

Time complexity: O(n) - where n is the number of nodes in the linked list. The algorithm iterates through the list twice: once to calculate the length and once to find the new head.

Space complexity: O(1) - constant space is used, as only a constant number of pointers are used regardless of the size of the input linked list.

Spiral Matrix IV (Leetcode-2326)

You are given two integers m and n, which represent the dimensions of a matrix.

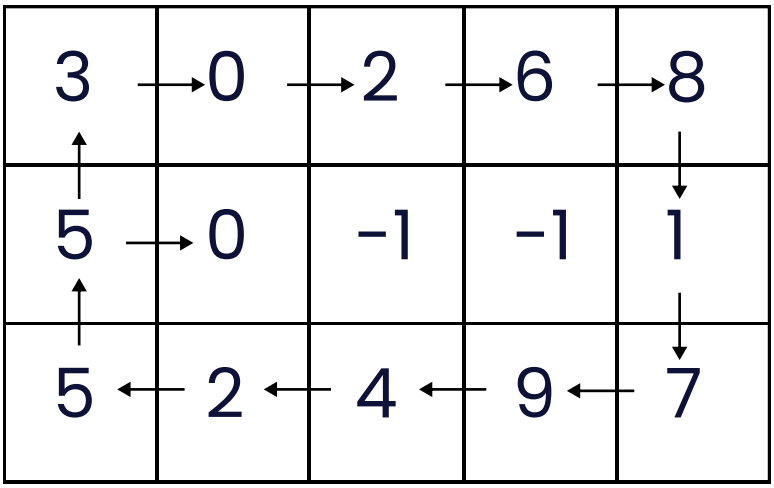
You are also given the head of a linked list of integers.

Generate an m x n matrix that contains the integers in the linked list presented in spiral order (clockwise), starting from the top-left of the matrix. If there are remaining empty spaces, fill them with -1.

Return the generated matrix.

Input: m = 3, n = 5, head = [3,0,2,6,8,1,7,9,4,2,5,5,0]
Output: [[3,0,2,6,8],[5,0,-1,-1,1],[5,2,4,9,7]]

Explanation: The diagram above shows how the values are printed in the matrix.
Note that the remaining spaces in the matrix are filled with -1.



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def spiralMatrix(self, m: int, n: int, head: ListNode) →
list[list[int]]:
        matrix = [[-1] * n for _ in range(m)]
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        row, col, d = 0, 0, 0

        while head:
            matrix[row][col] = head.val
 head = head.next
            next_row, next_col = row + directions[d][0], col +
directions[d][1]

            if not (0 ⩽ next_row < m and 0 ⩽ next_col < n and
matrix[next_row][next_col] == -1):
                d = (d + 1) % 4
                next_row, next_col = row + directions[d][0], col
+ directions[d][1]

            row, col = next_row, next_col

        return matrix
```

Time complexity: O(m * n) - where m is the number of rows and n is the number of columns in the resulting matrix. The algorithm iterates through each cell in the matrix to fill it with values from the linked list.

Space complexity: O(m * n) - the space used by the result matrix. The space complexity is determined by the size of the output matrix, which is m x n.

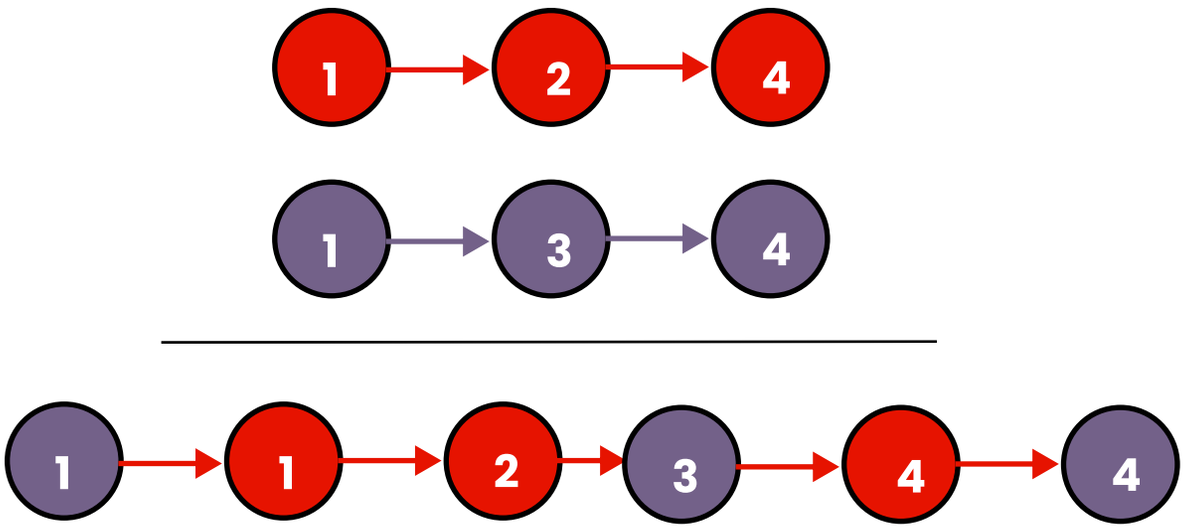
Merge 2 sorted lists (Leetcode-21)

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def mergeTwoLists(self, list1: ListNode, list2: ListNode) →
ListNode:
        if not list1:
            return list2
        if not list2:
            return list1

        if list1.val < list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

Time complexity: O(m + n) where m and n are the lengths of the two linked lists, l1 and l2. The function recursively traverses through each node in the linked lists.

Space complexity: O(m + n) due to the recursive stack. In the worst case, the maximum depth of the recursion would be the length of the longer linked list between l1 and l2.

Merge k sorted lists (Leetcode-23)

You are given an array of k linked-lists lists, each linked list is sorted in ascending order.
Merge all the linked lists into one sorted linked list and return it.

Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked lists are:
[
 1->4->5,
 1->3->4,
 2->6
]
merging them into one sorted list:
1->1->2->3->4->4->5->6

Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
self.val = val
        self.next = next

class Solution:
    def mergeKLists(self, lists: list[ListNode]) → ListNode:
        if not lists:
            return None
        return self.mergeKListsHelper(lists, 0, len(lists) - 1)

    def mergeKListsHelper(self, lists, start, end):
        if start == end:
            return lists[start]
        if start + 1 == end:
            return self.merge(lists[start], lists[end])
        mid = (start + end) // 2
        left = self.mergeKListsHelper(lists, start, mid)
        right = self.mergeKListsHelper(lists, mid + 1, end)
        return self.merge(left, right)
def merge(self, l1, l2):
        dummy = ListNode(0)
        curr = dummy

        while l1 and l2:
            if l1.val < l2.val:
                curr.next = l1
                l1 = l1.next
            else:
                curr.next = l2
                l2 = l2.next
            curr = curr.next

        curr.next = l1 if l1 else l2
        return dummy.next
```

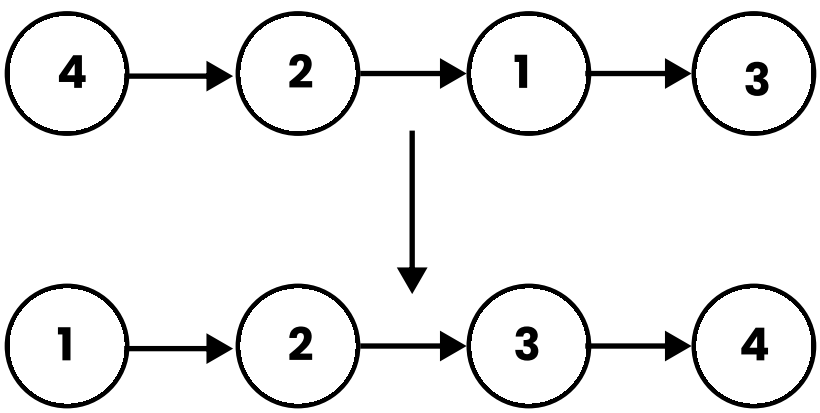
Time complexity: O(N log k), where N is the total number of nodes in all linked lists, and k is the number of linked lists. The mergeKLists function uses a divide-and-conquer strategy to merge the lists, and at each level of the recursion, it performs a linear-time merge operation.

Space complexity: O(log k), where k is the number of linked lists. This is the space used by the recursive call stack. In the worst case, the maximum depth of the recursion would be log k.

Sort List (Leetcode-148)

Given the head of a linked list, return the list after sorting it in ascending order.

Input: head = [4,2,1,3]
Output: [1,2,3,4]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        if not head or not head.next:
            return head

        # Find the middle of the list
        slow, fast = head, head
        prev = None
        while fast and fast.next:
            prev = slow
            slow = slow.next
            fast = fast.next.next

 # Split the list into two halves
        prev.next = None

        # Sort each half
        l1 = self.sortList(head)
        l2 = self.sortList(slow)
```

```
# Merge the sorted halves
        return self.merge(l1, l2)

    def merge(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
 current = dummy

        while l1 and l2:
            if l1.val < l2.val:
                current.next = l1
                l1 = l1.next
            else:
                current.next = l2
                l2 = l2.next
            current = current.next

        current.next = l1 if l1 else l2
        return dummy.next
```

Time complexity: O(n log n), where n is the number of nodes in the linked list. This is because the sortList function recursively divides the list into halves, and the merge function performs a linear-time merge operation at each level of the recursion.

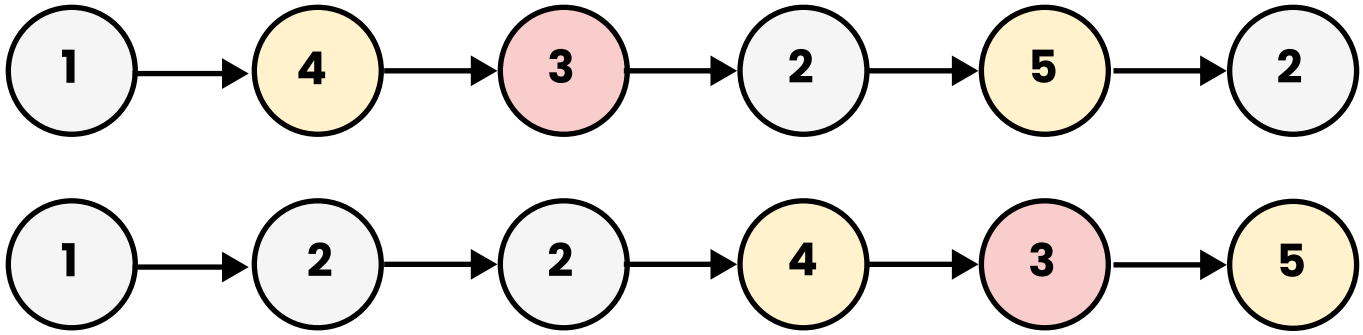
Space complexity: O(log n), where n is the number of nodes in the linked list. This is the space used by the recursive call stack. In the worst case, the maximum depth of the recursion would be log n.

Partition List (Leetcode-86)

Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x.

You should preserve the original relative order of the nodes in each of the two partitions.

Input: head = [1,4,3,2,5,2], x = 3
Output: [1,2,2,4,3,5]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def partition(self, head: ListNode, x: int) → ListNode:
        if not head:
            return head

dummyHeadSml = ListNode(0)
        dummyHeadEg = ListNode(0)
        curSmlNode = dummyHeadSml
        curEgNode = dummyHeadEg

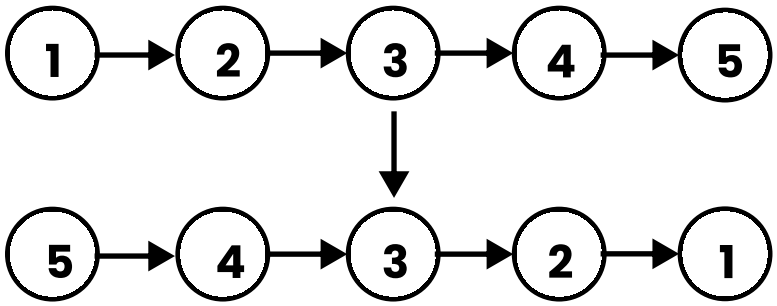
        current = head
        while current:
            next_node = current.next
            current.next = None
            if current.val < x:
                curSmlNode.next = current
                curSmlNode = current
            else:
                curEgNode.next = current
                curEgNode = current
            current = next_node
curSmlNode.next = dummyHeadEg.next
        return dummyHeadSml.next
```

Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm iterates through each node once and performs constant-time operations.

Space complexity: O(1). The algorithm uses a constant amount of extra space to store the left and right partitions, regardless of the size of the input linked list.

Reverse Linked List (Leetcode-206)

Given the head of a singly linked list, reverse the list, and return the reversed list.



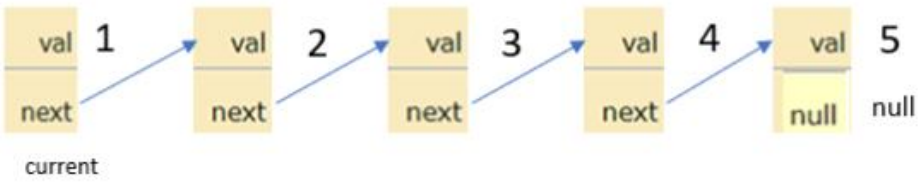
Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

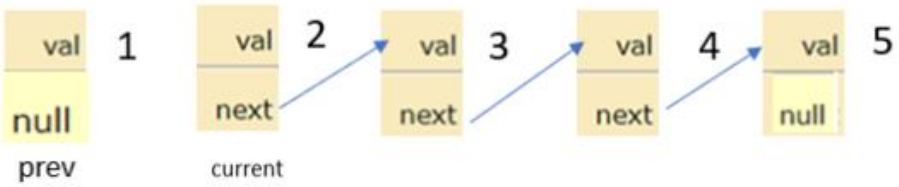
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        prev = None
        current = head

        while current:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node

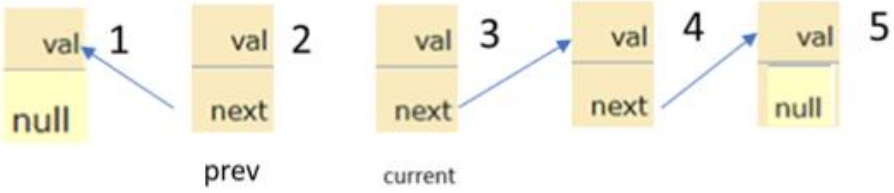
        return prev
```



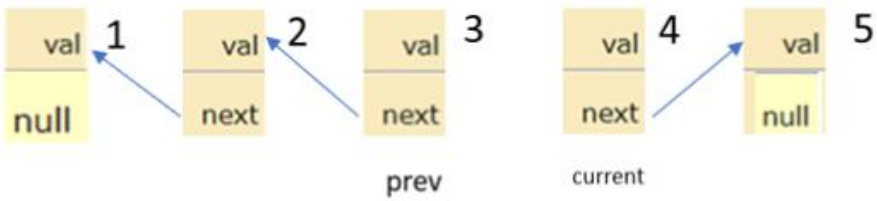
initially



After 1st iteration



After 2nd iteration



After 3rd iteration

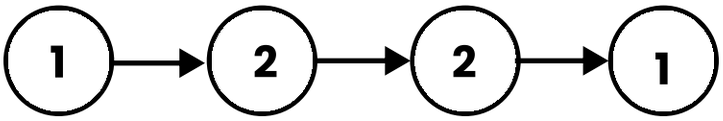
Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm iterates through each node once, performing constant-time operations in each iteration.

Space complexity: O(1). The algorithm uses a constant amount of extra space for the three-pointers (prev, current, next), regardless of the size of the input linked list.

Palindrome Linked List (Leetcode-234)

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Input: head = [1,2,2,1]
Output: true



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
self.val = val
        self.next = next

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        reverse_head = self.reverse(self.copy(head))

        while head and reverse_head:
            if head.val != reverse_head.val:
                return False
            head = head.next
            reverse_head = reverse_head.next

        return True

    def reverse(self, node: ListNode) -> ListNode:
        prev = None
        current = node

while current:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node

        return prev

    def copy(self, node: ListNode) -> ListNode:
        new_head = ListNode(node.val)
        current = new_head
        node = node.next

        while node:
            current.next = ListNode(node.val)
            node = node.next
            current = current.next
 return new_head
```

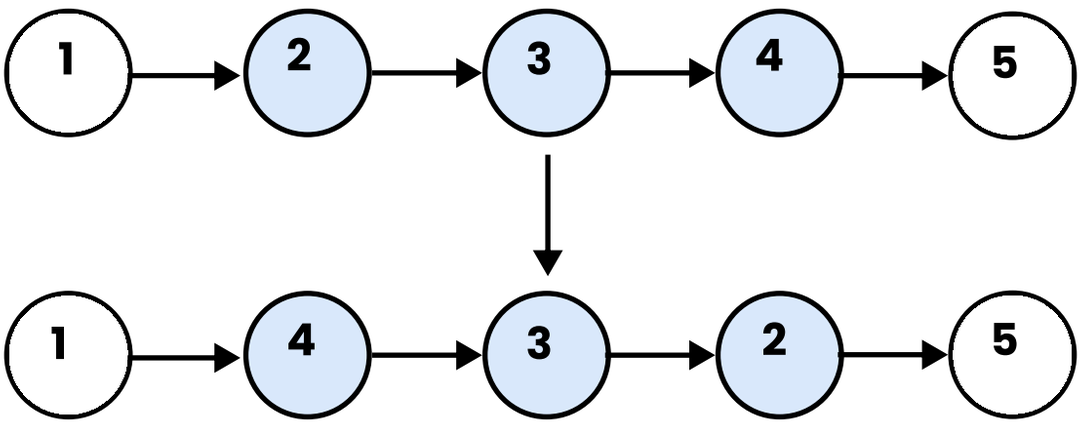
Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm iterates through the linked list twice—once to find the middle and reverse the second half, and once to compare the reversed second half with the first half.

Space complexity: O(1). The algorithm uses a constant amount of extra space for pointers and temporary variables, regardless of the size of the input linked list.

Reverse Linked List II (Leetcode-92)

Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.

Input: head = [1,2,3,4,5], left = 2, right = 4
Output: [1,4,3,2,5]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseBetween(self, head: ListNode, left: int, right:
int) → ListNode:
        if not head or left == right:
            return head

        dummy = ListNode(0)
        dummy.next = head
        prev = dummy
for _ in range(left - 1):
            prev = prev.next

        current = prev.next
        for _ in range(right - left):
            next_node = current.next
            current.next = next_node.next
            next_node.next = prev.next
            prev.next = next_node
return dummy.next
```

Time complexity: O(n), where n is the number of nodes in the linked list. The algorithm iterates through the linked list once, reversing the specified portion of the list.

Space complexity: O(1). The algorithm uses a constant amount of extra space for pointers and temporary variables, regardless of the size of the input linked list.

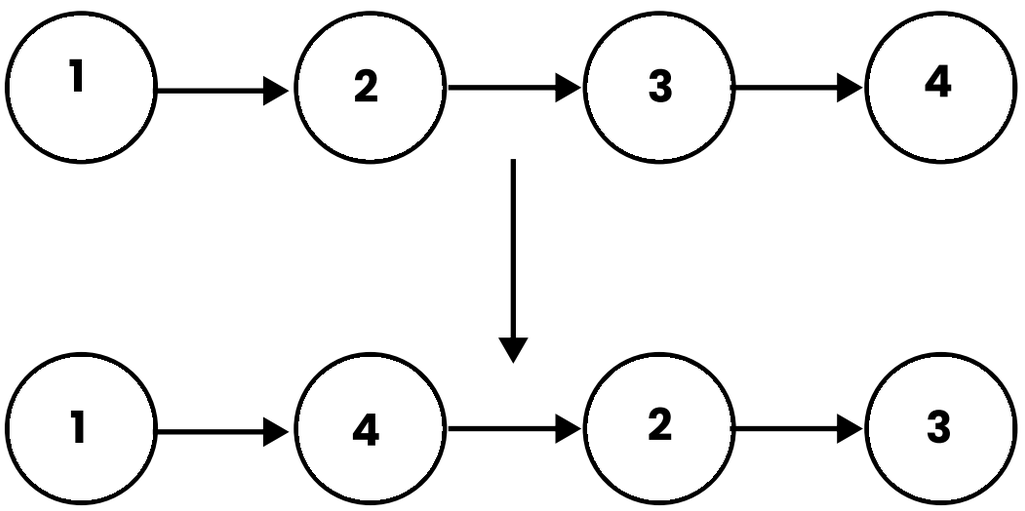
Reorder List (Leetcode-143)

You are given the head of a singly linked-list. The list can be represented as:

L0 → L1 → … → Ln - 1 → Ln
Reorder the list to be on the following form:

L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Input: head = [1,2,3,4]
Output: [1,4,2,3]



Code:

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: ListNode) -> None:
        if not head:
            return

        # Find the middle of the list

 slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
```

```
# Reverse the second half of the list
        prev, current = None, slow
        while current:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node

        # Merge the two halves
        first, second = head, prev
        while second.next:
            temp1, temp2 = first.next, second.next
            first.next = second
            second.next = temp1
            first, second = temp1, temp2
```

Introduction to Linked list

Assignment Solutions



PW SKILLS

1. In a singly linked list, deletion of data requires modification of how many pointers?

1
2
3
Depends upon the node being deleted.

Ans: It depends upon the node being deleted. Specifically, for the most common cases:

- Deleting the head node requires modifying 1 pointer (the head pointer).
- Deleting any other node requires modifying 1 pointer (the next pointer of the previous node to point to the next node of the node being deleted).

2. Predict the output for linked list = 1->2->3->4->5:

```
def traverse(head):
while head is not None and head.next is not None:
 print(head.data, end=" ")
     head = head.next.next
```

1 2 3 4 5
1 3 5
2 4
1 3

Ans: 1 3 5

Explanation: The `traverse` function iterates over the linked list, printing the data of the current node, and then moving two steps forward (`head = head.next.next`). This skips every second node.

3. Implement a Linked List class.
The user-defined LL should have insert (head,tail,idx) , delete(head,tail,idx) , get(idx) and display functions.

Ans:

```
class ListNode:
    def __init__(self, data=0, next=None):
        self.data = data
        self.next = next

class LinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert(self, idx, data):
        new_node = ListNode(data)
        if idx == 0:  # Insert at head
            new_node.next = self.head
            self.head = new_node
            if self.tail is None:
                self.tail = new_node
            return
```

```
current = self.head
        count = 0
        while current and count < idx - 1:
            current = current.next
            count += 1

        if current is None:
            if self.tail is None:
                self.head = new_node
            else:
                self.tail.next = new_node
            self.tail = new_node
        else:
            new_node.next = current.next
            current.next = new_node
            if new_node.next is None:
                self.tail = new_node

    def delete(self, idx):
        if self.head is None:
            return

        if idx == 0:  # Delete head
            self.head = self.head.next
            if self.head is None:
                self.tail = None
            return

        current = self.head
        count = 0
        while current.next and count < idx - 1:
            current = current.next
            count += 1

        if current.next:
            current.next = current.next.next
            if current.next is None:
                self.tail = current

    def get(self, idx):
        current = self.head
        count = 0
        while current and count < idx:
            current = current.next
            count += 1

        if current is None:
            return None
        return current.data

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" → ")
            current = current.next
        print("None")

# Usage example
ll = LinkedList()
ll.insert(0, 1)  # Insert 1 at head
ll.insert(1, 2)  # Insert 2 at tail
ll.insert(2, 3)  # Insert 3 at tail
ll.insert(1, 4)  # Insert 4 at index 1
ll.display()  # Expected Output: 1 → 4 → 2 → 3 → None

print("Element at index 2:", ll.get(2))  # Expected Output: 2

ll.delete(1)  # Delete element at index 1
ll.display()  # Expected Output: 1 → 2 → 3 → None

ll.delete(0)  # Delete head
ll.display()  # Expected Output: 2 → 3 → None
```